Documento de Especificación de Requisitos de Software (SRS)

Proyecto: Sistema de BANG Multijugador (V2.2 - Economía Virtual y Pagos CO)

Estándar de referencia: Adaptado de IEEE 830 / ISO/IEC/IEEE 29148 **Versión del documento:** 2.2.1 **Fecha:** 18 de Octubre de 2026 **Estado:** Actualizado – Post-Estabilización **Sistema objetivo:** Aplicación Web de BANG Multijugador Híbrida (`bingo_mod-2.2`)

Historial de cambios

Versión	Fecha	Descripción
2.2.0	14/10/2026	Línea base: economía Payplay, KYC por niveles, pagos CO (Wompi/ePayco).
2.2.1	18/10/2026	Estabilización del ciclo de juego y la economía: botón BINGO activo al completar línea (RF-24); cancelación de sala con reembolso total a todos los jugadores (RF-23); integridad bajo concurrencia con `flock` (RF-25); §3.2 y §6 alineados al sistema real; NFR y constantes corregidos; purga de escrow huérfano (§10).
2.2.2	19/10/2026	Interfaz de Billetera orientada a fichas: recarga y retiro por número de fichas (el monto COP pasa a ser valor derivado de N × 10.000); Libro Mayor mostrado en fichas; tarjeta única de Saldo Disponible (la retención escrow queda como dato secundario); checkout Wompi simulado evidencia las fichas a acreditar.

1. Introducción

1.1 Propósito

El presente documento define los requisitos para la versión **2.2.2** del sistema **BINGO Multijugador**. La plataforma opera como ecosistema transaccional con **modo Payplay** (apuestas con dinero real convertido a fichas virtuales), **sistema KYC** de verificación por niveles y un modelo híbrido de estado (JSON en disco + RDBMS ACID). La revisión 2.2.1 estabiliza el ciclo completo de partida — creación, lobby, cancelación, juego, validación de BINGO y liquidación de premios — garantizando integridad económica y comportamiento determinista ante concurrencia. La revisión 2.2.2 rediseña la interacción de la Billetera para operar directamente en fichas, eliminando la ambigüedad de montos COP en la entrada del usuario.

1.2 Alcance del Sistema

El sistema soporta dos modalidades simultáneas:

- **Modo Gratuito (Free-to-play):** Salas efímeras sin registro obligatorio, creadadas mediante alias. Cada sesión de invitado recibe una billetera de sesión demo con **5 fichas** para pruebas.

- **Modo Payplay:** Salas con costo de entrada (*Buy-in*) que generan un pozo de premios (*Prize Pool*). Requiere registro, saldo en billetera y opera bajo un modelo de integridad transaccional (RDBMS) combinado con retención temporal de fondos (*Escrow*).

El ciclo de vida completo de una sala incluye su **cancelación por el anfitrión antes de iniciar la partida**, con **reembolso total e inmediato** del escrow a todos los participantes (RF-23).

1.3 Definiciones, Acrónimos y Abreviaturas

- **Ficha / Token:** Unidad de valor de la plataforma. **1 Ficha = $10,000 COP** (pesos colombianos).
- **Buy-in:** Costo de entrada a una sala Payplay, expresado en fichas.
- **Prize Pool (Pozo):** Suma de Buy-ins menos la comisión, expresada en fichas.
- **Rake:** Comisión de la plataforma por organizar la partida (requisito: 10 %; ver deriva §9.1).
- **Escrow (Saldo Retenido):** Estado temporal de los fondos de un jugador mientras participa en una sala Payplay, garantizando que tiene fondos sin haberlos consumido aún.
- **Escrow por sala:** Marcador por código de sala en la sesión (`$_SESSION['escrow_sala']`) que registra cuánto retiene cada sala en esa sesión, permitiendo reembolsos exactos e idempotentes por sala.
- **Auto-reembolso:** Mecanismo por el cual la sesión del propio jugador recupera su escrow al detectar que su sala fue cancelada o eliminada (invitados y carreras).
- **Idempotencia:** Propiedad por la que repetir una operación (cancelar, reclamar reembolso, cantar BINGO) produce el mismo resultado económico sin pagos duplicados.
- **Flock:** Candado a nivel de archivo (`flock`) usado por `modificarSala()` para hacer atómicas las operaciones sobre el JSON de la sala ante polls concurrentes.
- **KYC (Know Your Customer):** Proceso de verificación de identidad para prevención de fraude y lavado de activos.
- **Webhook:** Petición HTTP POST enviada por la pasarela de pagos al servidor para notificar el estado de una transacción.

---

2. Descripción General del Producto

2.1 Arquitectura Híbrida (Estado y Transacciones)

Para balancear la alta frecuencia de actualización del juego en vivo (poll de 2.5 s) con la rigidez necesaria para manejar dinero real, el sistema utiliza un enfoque híbrido:

```
[Jugador Payplay] ──HTTP/SESIÓN──▶ [ Servidor Web PHP 8.2 ]
[Jugador Free]    ──HTTP─────────▶          │
                                            ├── Candado flock (sección crítica)
                                            │        └──▶ data/room_XXXX.json  (estado de
juego)
                                            ├── Transacciones PDO/ACID
                                            │        └──▶ MySQL: billeteras / transacciones /
salas
                                            └── Pasarelas (pendiente, ver §3.2)
                                                     └──▶ Wompi / ePayco
```

- **JSON de sala:** fuente de verdad del estado de juego (jugadores, cartones, balotas, estado). Cada escritura pasa por `modificarSala()` con `flock`.
- **RDBMS:** fuente de verdad del dinero (billeteras, transacciones, salas, ganadores). Toda escritura de saldo usa funciones centralizadas dentro de transacciones PDO.

2.2 Niveles de Verificación de Usuario (KYC)

- **Nivel 0 (Básico):** Registro con Email/Password. Acceso al modo Gratuito.
- **Nivel 1 (Identidad Verificada):** Confirmación vía OTP por email. Habilita recargas y participación en salas Payplay.
- **Nivel 2 (Financiero / Retiros):** Requiere carga de documento (CC/CE) y certificación bancaria. Obligatorio para solicitar retiros de ganancias.

---

3. Requisitos de Interfaces Externas

3.1 Interfaces de Usuario (UI/UX)

1. **Pantalla de Inicio (`index.php`):**
   - Toggle/Switch para seleccionar Modo Gratuito o Modo Payplay al crear sala.
   - En Payplay, incluye un selector de Buy-in en fichas (1, 2, 5, 10 fichas).
   - Muestra el saldo en fichas con equivalencia en COP.
2. **Panel de Usuario y Billetera (`wallet.php`):**
   - Indicador de Saldo: tarjeta única de **Saldo Disponible** en fichas con equivalencia en COP; la retención escrow se presenta como dato secundario de la misma tarjeta.
   - Módulo de Recarga: entrada por **número de fichas** (1/5/10/20, mínimo 1) con equivalente COP calculado en vivo; el checkout simulado de Wompi/ePayco evalúa las fichas a acreditar y el monto COP derivado (N × 10,000).
   - Módulo de Retiros: solicitud **por número de fichas**; formulario bloqueado si el usuario no es Nivel 2.
   - Libro Mayor de movimientos contables mostrado en fichas (columna "Monto (Fichas)").
3. **Panel de Verificación (`perfil.php`):**
   - Interfaz para subir documentos KYC y validar token OTP.
4. **Sala de Espera / Lobby (`espera.php`):**
   - Visor en vivo del Pozo Acumulado en fichas (modo Payplay).
   - Controles del host: velocidad de balotas (5/10/15 s), nomenclatura (normal/reducida), botón **Iniciar partida**.
   - **Nuevo (2.2.1):** botón **"Cancelar sala"** visible solo para el host, con confirmación en dos toques ("¿Reembolsar a todos y cancelar? Toca de nuevo"). Muestra el resultado: número de jugadores reembolsados y total de fichas devueltas.

- **Nuevo (2.2.1):** cuando la sala pasa a `cancelado`, todos los jugadores ven el aviso "Sala cancelada – Tus fichas fueron reembolsadas" y se auto-reembolsa su escrow antes de salir al inicio.

5. **Durante la Partida (`juego.php`):**
   - Visor de balotas, cartón marcado en vivo y panel de jugadores.
   - **Botón "¡BINGO!":** permanece **inactiva** mientras no se complete una línea; se **activa (habilitada y con resaltado)** automáticamente cuando el cliente detecta fila, columna o diagonal completa. El servidor valida de forma autoritativa antes de pagar (RF-24).
   - Confirmación en dos toques para abandonar la partida en curso.

3.2 Endpoints (API REST)

Endpoint	Método	Acción	Estado (2.2.1)
`php/auth/registro.php`	`POST`	Registro Nivel 0 + creación de billetera y KYC.	Implementado
`php/auth/login.php`	`POST`	Login email/contraseña e inicio de sesión.	Implementado
`php/auth/logout.php`	`POST`	Cierre de sesión.	Implementado
`php/auth/google.php`	`GET/POST`	Login/registro con Google OAuth.	Implementado
`php/auth/verificar.php`	`POST`	Validación OTP para ascenso a Nivel 1.	**Pendiente**
`php/crear_sala.php`	`POST`	Crea sala free/payplay; retiene escrow (sesión + BD).	Implementado
`php/unirse_sala.php`	`POST`	Alta atómica de jugador bajo candado; retiene escrow.	Implementado
`php/iniciar_partida.php`	`POST`	Host inicia: consolida pozo, genera cartones, `jugando`.	Implementado
`php/verificar_estado.php`	`GET`	Estado de sala (poll 2.5 s) y cartón del jugador.	Implementado
`php/verificar_estado.php`	`POST`	Acciones: `marcar`, `salir`, **`reembolso_cancelacion`**.	Implementado (`reembolso_cancelacion` nuevo)
`php/cantar_bingo.php`	`POST`	Valida y paga un BINGO de forma idempotente.	Implementado (2.2.1)
`php/cancelar_sala.php`	`POST`	Cancela la sala pre-inicio con reembolso total a todos.	**Nuevo (2.2.1)**
`php/cambiar_velocidad.php`	`POST`	Ajusta intervalo de balotas (solo host).	Implementado
`php/billetera_acciones.php`	`POST`	Operaciones de billetera y estado KYC.	Implementado
`php/wompi_simulado.php`	`POST`	Simulador de pasarela para pruebas.	Implementado (simulador)
`php/pagos/checkout.php`	`POST`	Firma de petición y hash para widget de pago.	**Pendiente**
`php/pagos/webhook.php`	`POST`	Recibe confirmación de la pasarela, valida firma y acredita fichas.	**Pendiente**

Endpoint	Método	Acción	Estado (2.2.1)
`php/pagos/retiro.php`	`POST`	Crea solicitud de retiro (Requiere Nivel 2).	**Pendiente**

---

4. Requisitos Funcionales

Módulo 1: Billetera y Pagos (Integración ePayco/Wompi)

- **RF-14: Recargas por Webhook.** La recarga se solicita en **número de fichas** (mínimo 1; la interfaz ofrece atajos de 1/5/10/20). El sistema deriva el monto en COP (N × 10,000, con 1 Ficha = $10,000 COP) que presenta la pasarela para el cobro. Al recibir un evento `APPROVED` de la pasarela, el sistema acredita las N fichas a `saldo_disponible`. Se conserva compatibilidad con la entrada antigua por monto COP (división entera con validación de mínimo 1 ficha).
- **RF-15: Validación Criptográfica (Anti-Fraude).** Todo webhook entrante valida su autenticidad reconstruyendo el Hash/Checksum (SHA-256) con la Llave Secreta de Eventos mediante `hash_equals()`.
- **RF-16: Idempotencia de Transacciones.** El sistema verifica en la base de datos que la `referencia_pasarela` no exista antes de procesar el saldo para prevenir el doble gasto.
- **RF-17: Solicitud de Retiros.** Usuarios Nivel 2 pueden solicitar retiros indicando el **número de fichas** (mínimo 1, tope del saldo disponible). El sistema deriva el montante (Fichas × $10,000 COP) y descuenta las fichas de `saldo_disponible`. La solicitud pasa a estado `PENDIENTE` para desembolso manual.

Módulo 2: Mecánica de Apuestas (Ciclo Escrow)

- **RF-18: Retención (Fase 1).** Al crear o unirse a una sala Payplay, el Buy-in se debita de `saldo_disponible` y se acredita a `saldo_retenido` en **dos vías simultáneas**: en la base de datos (usuarios con cuenta) y en la sesión (con marcador por sala `escrow_sala[id_sala]`). La retención es idempotente por topes (`min(monto, saldo_retenido)`).
- **RF-19: Cobro de Buy-in (Fase 2).** Al iniciar la partida, el Host desencadena la consolidación: el `pozo_total` de la sala se fija en `jugadores × buy_in` (JSON) y en BD vía el trigger `trg_jugador_sala_insert`; la sala pasa a `estado_financiero = COBRADA`; cada jugador recibe su cartón generado por el servidor. *(Estado: **parcial** — la liberación automática de `billeteras.saldo_retenido` al consolidar está pendiente; ver §9.2.)*
- **RF-20: Reembolso (Fase 3).** Si un jugador abandona el lobby, su Buy-in retorna a `saldo_disponible` (sesión y BD) y baja el pozo. Si la sala se cancela antes de arrancar, el sistema hace **reembolso masivo total a todos los jugadores** (mecánico completo en RF-23).
- **RF-21: Reparto de Premios (Fase 4).** Al confirmarse un BINGO válido: $$Pozo Neto = (Participantes \times BuyIn) \times (1 - Rake)$$ *(Rake requisito = 10 %)*. Si hay un empate en la misma balota, el Pozo Neto se divide equitativamente. El premio se transfiere automáticamente al `saldo_disponible` del ganador mediante transacción `PREMIO`, y la sala queda en `estado_financiero = REPARTIDA`.

- **RF-22: Límite y Desconexión.** Se permite estrictamente 1 cartón por jugador por sala Payplay. Si el jugador se desconecta en curso, su cartón se evalúa en el servidor de forma automática; si llega a cantar, el pago se emite a la billetera de su cuenta.
- **RF-23: Cancelación de Sala (NUEVO – 2.2.1).** Reembolso total e inmediato del escrow cuando la sala se cancela **antes de iniciar**:
  1. **Autoría:** sólo el host puede cancelar (`host_id` de la sesión) y sólo con `estado = 'esperando'`. En la UI se exige confirmación en dos toques.
  2. **Alcance del reembolso — a TODOS los jugadores:** los usuarios con cuenta reciben su Buy-in completo en `saldo_disponible` (BD) **en la misma petición**; el host además recibe su reembolso de sesión; los invitados reciben su reembolso de sesión mediante el auto-reembolso (`reembolso_cancelacion`) que dispara su lobby al detectar la cancelación.
  3. **Red de seguridad:** si la sesión del invitado detecta la cancelación cuando la sala ya fue borrada, el poll de estado ejecuta el auto-reembolso contra el marcador `escrow_sala` de esa sesión.
  4. **Idempotencia triple:** bandera `escrow_reembolsado` en la sala, marcador por sala en sesión y tope `min(monto, saldo_retenido)` en BD. Repetir la operación nunca paga dos veces.
  5. **Efectos:** la sala pasa a `estado = 'cancelado'`, `pozo_total = 0`, `estado_financiero = 'REEMBOLSADA'`, se fija `finalizado_at` (dispara la auto-limpieza a 60 s) y se sincroniza el estado con BD.
  6. **Concurrencia:** la validación y los reembolsos ocurren atómicamente bajo candado (`modificarSala`); unirse a una sala que se está cancelando queda rechazado sin retener fondos.
- **RF-24: Validación y Pago de BINGO (NUEVO – 2.2.1).**
  1. **Cliente (UX):** `tieneLineaCompleta()` espeja exactamente al `validarBingo()` del servidor; el botón "¡BINGO!" se activa únicamente al completar fila, columna o diagonal.
  2. **Servidor autoritativo:** toda marcada del cartón debe existir en el `historial` de la sala (anti-trampa) y debe existir combinación ganadora. Un reclamo sin línea válida se rechaza con mensaje claro.
  3. **Atómico e idempotente:** validación y registro del ganador ocurren en una sección crítica con `flock`; en BD se ejecuta **una sola transacción** cuyo **INSERT en `ganadores` va primero**, aprovechando la restricción única `uq_ganadores_sala` como guardia de idempotencia (un solo ganador por sala).
  4. **`carton_id`:** debe ser un id real de la tabla `cartones` (nunca un `jugadores_sala.id`), evitando violaciones de la FK `fk_ganadores_carton`.
  5. **Robustez:** si la BD falla, se hace `rollback` y se registra en log; el resultado del juego (JSON) se conserva y **siempre** se responde JSON (sin HTTP 500 silencioso). Un segundo reclamo rechaza con "La partida no está activa" y la sala queda `finalizado`.
- **RF-25: Integridad bajo Concurrencia (NUEVO – 2.2.1).**
  1. Toda escritura del JSON de sala (estado, poll/generación de balotas, marcar, salir, cantar, unirse, cancelar) pasa por `modificarSala()` con `flock`, eliminando condiciones de carrera entre polls.
  2. El alta de jugadores revalida `estado = 'esperando'` y capacidad **bajo candado**, y revierte la retención de escrow si el alta falla.

3. El dinero se modifica únicamente en funciones centralizadas (`retenerSaldoEscrow` , `reembolsarSaldoEscrow` , `reembolsarSaldoEscrowBD` , `acreditarPremio*`) dentro de transacciones PDO (`begin/commit/rollback`).
4. Los reembolsos son idempotentes por topes: operaciones repetidas o concurrentes no pueden duplicar dinero.

Ciclo de Vida de la Sala

```
esperando ──(host: iniciar)──▶ jugando ──(BINGO válido)──▶ finalizado
    │                            │
    │                            └──(todos ausentes)──▶ cancelado
    └──(host: cancelar)──────────────────────────────▶ cancelado

finalizado / cancelado ──(60 s tras finalizado_at)──▶ eliminación automática
                                                      (+ reembolsos protegidos
                                                        contra doble pago)
```

---

5. Requisitos No Funcionales (NFR)

- **S-05 (Bloqueo Transaccional):** El dinero sólo se modifica dentro de transacciones PDO con bloqueo consistente; el estado de la sala se modifica con candado de archivo (`flock`) en secciones críticas. Las operaciones de saldo son idempotentes por topes (`saldo_retenido`), por lo que reintentos y peticiones concurrentes no duplican fondos.
- **S-06 (Libro Mayor Append-Only):** La tabla `transacciones` es inmutable. No se permite `UPDATE` ni `DELETE` sobre registros históricos. Los errores y mantenimientos se corrigen con asientos nuevos (p. ej. `tipo = 'REEMBOLSO'` con referencia `PURGA-ESCROW-*`).
- **S-07 (Whitelisting IPs):** El endpoint `webhook.php` quedará restringido a nivel de servidor para aceptar únicamente tráfico desde IPs documentadas de Wompi o ePayco (pendiente de implementación, ver §3.2).
- **M-03 (Performance JSON):** La mecánica de marcado de casillas y extracción de balotas opera sobre el JSON de sala en disco con `flock` (sin base de datos relacional en la ruta caliente), garantizando respuestas menores a 80 ms durante el polling. No se usa Redis en el alcance actual.

---

6. Modelo de Datos (Esquema Relacional Implementado)

> Fuente de verdad: `database/schema.sql` . Este esquema refleja el sistema **realmente desplegado** en `bingo_db` (MySQL 8 / MariaDB, InnoDB, utf8mb4).

```
-- Usuarios
CREATE TABLE usuarios (
    id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    nickname VARCHAR(30) NOT NULL,
    email VARCHAR(255) NOT NULL,
```

```
    password_hash VARCHAR(255) NULL,            -- NULL si es cuenta Google
    google_id VARCHAR(255) NULL,
    nivel_verificacion TINYINT UNSIGNED NOT NULL DEFAULT 0,  -- 0, 1, 2
    rol ENUM('JUGADOR','ADMIN','MODERADOR') NOT NULL DEFAULT 'JUGADOR',
    estado ENUM('ACTIVO','SUSPENDIDO','INACTIVO') NOT NULL DEFAULT 'ACTIVO',
    creado_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    actualizado_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    CONSTRAINT uq_usuarios_email UNIQUE (email),
    CONSTRAINT uq_usuarios_nickname UNIQUE (nickname)
) ENGINE=InnoDB;

-- Verificación de identidad (KYC)
CREATE TABLE verificaciones_kyc (
    id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    usuario_id INT UNSIGNED NOT NULL,
    estado ENUM('PENDIENTE','EN_REVISION','APROBADO','RECHAZADO') NOT NULL DEFAULT 'PENDIENTE',
    nivel TINYINT UNSIGNED NOT NULL DEFAULT 0,   -- 0, 1, 2
    documento_tipo ENUM('CC','CE','TI','NIT') NULL,
    documento_num VARCHAR(20) NULL,
    otp_enviado VARCHAR(6) NULL,
    otp_verificado_at TIMESTAMP NULL,
    CONSTRAINT fk_kyc_usuario FOREIGN KEY (usuario_id)
        REFERENCES usuarios(id) ON DELETE CASCADE,
    CONSTRAINT uq_kyc_usuario UNIQUE (usuario_id)
) ENGINE=InnoDB;

-- Billeteras (saldo en fichas: disponible + escrow)
CREATE TABLE billeteras (
    id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    usuario_id INT UNSIGNED NOT NULL,
    saldo_disponible DECIMAL(12,2) NOT NULL DEFAULT 0.00,
    saldo_retenido DECIMAL(12,2) NOT NULL DEFAULT 0.00,   -- Escrow
    actualizado_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    CONSTRAINT fk_billeteras_usuario FOREIGN KEY (usuario_id)
        REFERENCES usuarios(id) ON DELETE CASCADE,
    CONSTRAINT uq_billeteras_usuario UNIQUE (usuario_id)
) ENGINE=InnoDB;

-- Libro mayor inmutable (sólo INSERT)
CREATE TABLE transacciones (
    id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    billetera_id INT UNSIGNED NOT NULL,
    tipo ENUM('RECARGA','BUY_IN','REEMBOLSO','PREMIO','RETIRO','BONO') NOT NULL,
    monto DECIMAL(12,2) NOT NULL,               -- + crédito / – débito
    saldo_resultante DECIMAL(12,2) NOT NULL,
    descripcion TEXT NULL,
    referencia VARCHAR(50) NULL,                -- ej. PURGA-ESCROW-3, PREMIO-8RJ7-...
    fecha TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_transacciones_billetera FOREIGN KEY (billetera_id)
        REFERENCES billeteras(id) ON DELETE CASCADE,
    INDEX idx_transacciones_fecha (fecha),
    INDEX idx_transacciones_tipo (tipo)
) ENGINE=InnoDB;

-- Salas de bingo (free / payplay)
CREATE TABLE salas (
```

```
    id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    codigo VARCHAR(4) NOT NULL,                 -- código único de 4 caracteres
    nombre VARCHAR(30) NOT NULL,
    tipo_sala ENUM('free','payplay') NOT NULL DEFAULT 'free',
    buy_in DECIMAL(12,2) NOT NULL DEFAULT 0.00,
    comision_pct DECIMAL(5,2) NOT NULL DEFAULT 10.00,   -- ver deriva §9.1
    pozo_total DECIMAL(12,2) NOT NULL DEFAULT 0.00,
    estado ENUM('esperando','jugando','finalizado','cancelado') NOT NULL DEFAULT 'esperando',
    host_id INT UNSIGNED NOT NULL,
    modo ENUM('normal','reducido') NOT NULL DEFAULT 'normal',
    intervalo_fichas TINYINT UNSIGNED NOT NULL DEFAULT 10,
    creado_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    iniciado_at TIMESTAMP NULL,
    finalizado_at TIMESTAMP NULL,
    CONSTRAINT fk_salas_host FOREIGN KEY (host_id) REFERENCES usuarios(id) ON DELETE CASCADE,
    CONSTRAINT uq_salas_codigo UNIQUE (codigo),
    INDEX idx_salas_estado (estado)
) ENGINE=InnoDB;

-- Participantes (ciclo escrow)
CREATE TABLE jugadores_sala (
    id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    sala_id INT UNSIGNED NOT NULL,
    usuario_id INT UNSIGNED NOT NULL,
    buy_in_pagado DECIMAL(12,2) NOT NULL DEFAULT 0.00,
    posicion TINYINT UNSIGNED NOT NULL,
    es_bot BOOLEAN NOT NULL DEFAULT FALSE,
    joined_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_jugadores_sala FOREIGN KEY (sala_id) REFERENCES salas(id) ON DELETE CASCADE,
    CONSTRAINT fk_jugadores_usuario FOREIGN KEY (usuario_id) REFERENCES usuarios(id) ON DELETE
CASCADE,
    CONSTRAINT uq_jugadores_sala UNIQUE (sala_id, usuario_id)
) ENGINE=InnoDB;

-- Cartones (matriz 5x5 + marcadas, sincronizados con la sala JSON)
CREATE TABLE cartones (
    id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    jugador_sala_id INT UNSIGNED NOT NULL,      -- FK real: NUNCA se usa como carton_id
    numero_carton TINYINT UNSIGNED NOT NULL DEFAULT 1,
    grid JSON NOT NULL,
    marcadas JSON NOT NULL,
    es_ganador BOOLEAN NOT NULL DEFAULT FALSE,
    creado_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_cartones_jugador FOREIGN KEY (jugador_sala_id)
        REFERENCES jugadores_sala(id) ON DELETE CASCADE
) ENGINE=InnoDB;

-- Historial de balotas cantadas
CREATE TABLE historial_fichas (
    id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    sala_id INT UNSIGNED NOT NULL,
    ficha VARCHAR(3) NOT NULL,                  -- A01, B02, C03...
    cantada_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_historial_sala FOREIGN KEY (sala_id) REFERENCES salas(id) ON DELETE CASCADE
) ENGINE=InnoDB;
```

```
-- Ganadores: MÁS UN ganador por sala (guardia de idempotencia del pago)
CREATE TABLE ganadores (
    id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    sala_id INT UNSIGNED NOT NULL,
    usuario_id INT UNSIGNED NOT NULL,
    carton_id INT UNSIGNED NULL,                -- id REAL de cartones (FK) o NULL
    premio_ganado DECIMAL(12,2) NOT NULL,
    fecha TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_ganadores_sala FOREIGN KEY (sala_id) REFERENCES salas(id) ON DELETE CASCADE,
    CONSTRAINT fk_ganadores_usuario FOREIGN KEY (usuario_id) REFERENCES usuarios(id) ON DELETE
CASCADE,
    CONSTRAINT fk_ganadores_carton FOREIGN KEY (carton_id) REFERENCES cartones(id) ON DELETE SET
NULL,
    CONSTRAINT uq_ganadores_sala UNIQUE (sala_id)
) ENGINE=InnoDB;

-- Solicitudes de retiro (Nivel 2)
CREATE TABLE solicitudes_retiro (
    id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    usuario_id INT UNSIGNED NOT NULL,
    fichas DECIMAL(12,2) NOT NULL,
    monto_cop DECIMAL(12,2) NOT NULL,
    banco VARCHAR(50) NOT NULL,
    tipo_cuenta ENUM('Ahorros','Corriente') NOT NULL DEFAULT 'Ahorros',
    num_cuenta VARCHAR(20) NOT NULL,
    titular VARCHAR(100) NOT NULL,
    estado ENUM('PENDIENTE','PROCESANDO','APROBADO','RECHAZADO') NOT NULL DEFAULT 'PENDIENTE',
    referencia VARCHAR(50) NULL,
    procesado_por INT UNSIGNED NULL,
    procesado_at TIMESTAMP NULL,
    fecha TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_retiros_usuario FOREIGN KEY (usuario_id) REFERENCES usuarios(id) ON DELETE
CASCADE
) ENGINE=InnoDB;
```

Elementos adicionales del modelo:

- **Estado de juego en JSON:** `data/room_<CODIGO>.json` guarda jugadores, cartones, historial de balotas, `host_id` aleatorio de sesión y el estado (`esperando/jugando/finalizado/cancelado`). Es la fuente de verdad del juego; la BD es la del dinero.
- **Trigger `trg_jugador_sala_insert`:** al insertar un participante en una sala Payplay, incrementa `salas.pozo_total` con su `buy_in_pagado`.
- **Procedimientos:** `sp_crear_usuario`, `sp_recargar_billetera`, `sp_retener_escrow`, `sp_acreditar_premio`, `sp_solicitar_retiro`.
- **Vistas:** `v_resumen_billeteras`, `v_salas_activas`, `v_transacciones_usuarios`.
- **Nota de deriva:** en producción `salas.comision_pct` existe como `DECIMAL(5,4) DEFAULT 0.0500` (5 %) — ver §9.1.

---

7. Paleta de Colores

Color	Hex	Uso
Azul Noche	`#0F172A`	Fondo general de la aplicación
Blanco	`#FFFFFF`	Tipografía principal, números del cartón
Gris Carbón	`#1E293B`	Fondos de tarjetas, superficies elevadas
Naranja Neón	`#F97316`	Botones principales, fichas marcadas, acentos
Rojo	`#EF4444`	Alertas, errores, acciones destructivas (p. ej. Cancelar sala)
Verde Brillante	`#10B981`	Acciones financieras, saldo, aprobaciones
Amarillo Dorado	`#F59E0B`	Premios, pozo acumulado, buy-in seleccionado

---

8. Constantes del Sistema

Constante	Valor	Descripción
`VALOR_FICHA`	10000	Valor de 1 ficha en COP
`FICHAS_BONO_BIENVENIDA`	0	Sin bono de bienvenida (el usuario recarga desde la app)
`FICHAS_DEMO_INVITADO`	5	Billetera de sesión demo para invitados (modo free)
`RAKE_PORCENTAJE`	0.10	Comisión de la plataforma — requisito (ver deriva §9.1)
`BUY_IN_MINIMO`	1	Mínimo de fichas para buy-in
`JUGADORES_MAX`	20	Máximo de jugadores por sala
`INTERVALO_FICHAS`	5 / 10 / 15	Segundos entre balotas (default 10; auto-reducido si < 5 jugadores)
`LONGITUD_CODIGO_SALA`	4	Caracteres del código único de sala
`POLL_INTERVALO_MS`	2500	Intervalo de polling de lobby/juego
`LIMPIEZA_SALA_SEG`	60	Segundos tras `finalizado_at` para auto-eliminación de la sala

---

9. Derivas Conocidas y Trabajo Pendiente

1. **Comisión (Rake):** el requisito y el DDL dicen 10 %, pero `salas.comision_pct` en producción quedó como `DECIMAL(5,4) DEFAULT 0.0500` (5 %). **Pendiente:** alinear la columna/trigger al 10 % (o aprobar documentalmente el 5 % y corregir el SRS).
2. **Consolidación de escrow en BD (RF-19 fase 2):** `iniciar_partida` consolida el pozo (JSON + trigger) pero no pone `billeteras.saldo_retenido` en cero, y la limpieza de una sala ya finalizada puede reembolsar Buy-ins gastados. **Pendiente:** automatizar la liberación del escrow al consolidar/finalizar y

evitar el reembolso de Buy-ins de partidas ya cobradas. *(Compensado manualmente con la purga del 18/10/2026, §10.)*

3. **Pasarelas de pago y OTP:** `php/pagos/*` y `php/auth/verificar.php` siguen pendientes (existe `wompi_simulado.php` para pruebas).
4. **Invitados en Payplay:** el requisito exige registro para salas Payplay, pero hoy un invitado puede crear/unirse con sus fichas demo de sesión. **Pendiente:** restringir el modo Payplay a usuarios con cuenta.
5. **Saldos históricos inflados:** usuarios 4, 8, 9 y 11 acumulan saldo extra por el bug histórico de pagos duplicados de premio (~90 transacciones PREMIO). **Pendiente:** reversión contable con asientos de reversión.

---

10. Operación y Mantenimiento de Datos

- **18/10/2026 – Purga de escrow huérfano:** se eliminaron **31.00 fichas** retenidas en **7 billeteras** correspondientes a salas ya inexistentes, dejando `saldo_retenido = 0` en todas las billeteras para reinicio limpio de pruebas. Cada ajuste quedó auditado en `transacciones` (`tipo = 'REEMBOLSO'`, `referencia = PURGA-ESCROW-<billetera_id>`), respetando el libro mayor append-only. En el mismo mantenimiento se eliminaron 7 salas fantasma en BD (con sus filas de `jugadores_sala`, `cartones`, `historial_fichas`) y 39 archivos JSON de salas obsoletas.